

POS TECH

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 4 | AULA 05 -

CONCEITOS DE PROGRAMAÇÃO IMPERATIVA E REATIVA

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3

HANDS ON.....4

SAIBA MAIS 5

O QUE VOCÊ VIU NESTA AULA?.....9

REFERÊNCIAS 10

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Nesta aula, aprenderemos sobre programação imperativa e reativa, explorando seus conceitos, diferenças e casos de uso. Veremos como a programação imperativa se baseia em um fluxo sequencial de comandos, enquanto a programação reativa é orientada a eventos, focando em trabalhar com fluxos de dados de forma assíncrona e responsiva.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Nesta videoaula, nós abordaremos exemplos práticos demonstrando a programação reativa e imperativa.

Mas antes de você assistir as videoaulas, seguem todos os links utilizados durante as explicações:

https://github.com/FIAP/JAVA_QUARKUS

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

Conforme vimos anteriormente, o **Quarkus** é uma plataforma versátil que permite o desenvolvimento de aplicações Java otimizadas para a nuvem, suportando tanto a **programação imperativa** quanto a **reativa**. Essa flexibilidade oferece aos desenvolvedores e desenvolvedoras a liberdade de escolher o paradigma que melhor se adapta ao contexto de sua aplicação, sem comprometer o desempenho ou a escalabilidade.

Ainda nesta aula, exploraremos como o Quarkus implementa esses paradigmas e quando usar cada um para maximizar a eficiência do seu projeto.Mas, antes, vejamos o que é a programação imperativa e reativa.

A **programação imperativa** é um dos paradigmas mais antigos e amplamente utilizados no desenvolvimento de software. Nesse modelo, programadores escrevem um conjunto de instruções claras e sequenciais para o computador executar. O foco está em **como**a tarefa deve ser realizada, detalhando cada passo necessário para atingir o objetivo final.

Esse paradigma é semelhante à ideia de seguir uma receita: você fornece uma lista de ações detalhadas que devem ser realizadas em uma ordem específica, manipulando diretamente os dados e alterando o estado do programa conforme necessário.

Para ficar mais claro com ela funciona, vejamos algumas das suas características principais:

1. **Sequência de comandos:**a execução do código segue uma sequência lógica e ordenada. O fluxo é definido explicitamente por quem programa, controlando cada etapa da operação.
2. **Controle explícito do fluxo:**o desenvolvedor ou desenvolvedora decide como o programa avança, utilizando estruturas como loops (for, while) e condicionais (if, else).
3. **Operações sincronizadas:** as ações geralmente são executadas de forma síncrona, uma de cada vez, seguindo a ordem definida.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

4. **Facilidade de depuração:** por ser um modelo explícito e direto, é relativamente fácil rastrear erros ao seguir o fluxo de execução.

Esse é o formato que a maioria dos programadores e programadoras já estão habituados a utilizar.

Imagine um cenário em que um servidor web atenda a múltiplas requisições de clientes simultaneamente. Se uma dessas requisições envolver uma operação de leitura de um arquivo grande em disco, o servidor ficará bloqueado esperando a conclusão dessa operação antes de poder atender a outras solicitações.

Esse cenário pode levar à ineficiência geral do sistema e ao baixo desempenho em momentos de alta carga. A figura a seguir ilustra esse assunto:

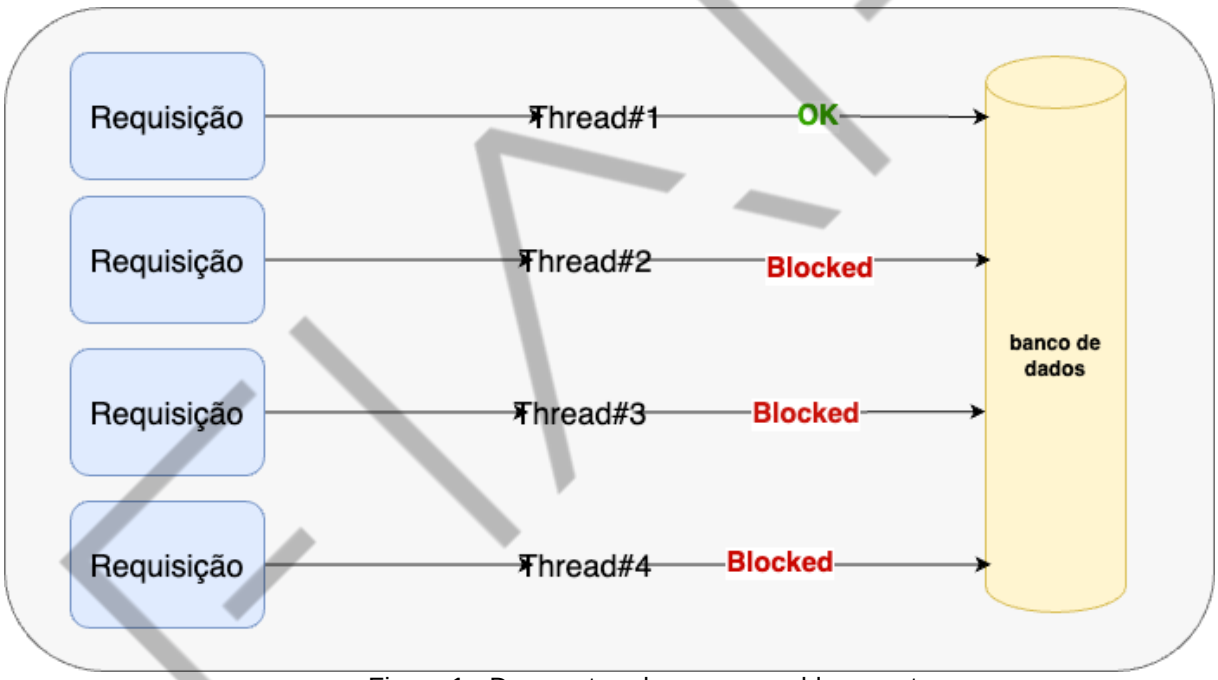


Figura 1 - Demonstrando processos bloqueantes
Fonte: elaborado pelo autor (2024)

Para entendermos essa imagem, imagine que temos uma aplicação desenvolvida em Java para um sistema X e estamos utilizando uma controller para que os usuários possam acessar o nosso sistema.

No lado esquerdo da imagem, temos as requisições que estão sendo realizadas; depois, temos as threads que são criadas para a nossa requisição. Em seguida, é realizada uma pesquisa no banco de dados, que bloqueia a thread até que ela seja finalizada.Assim que essa pesquisa for finalizada, a operação no banco de dados da thread retorna com os dados para a nossa requisição.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

No fluxo descrito anteriormente, fica evidente que enfrentamos um gargalo causado pelo processo bloqueante da thread durante a espera pela conclusão da pesquisa no banco de dados.

Esse gargalo resulta em um processo ocioso, no qual a thread fica inativa e é incapaz de realizar outras tarefas úteis enquanto espera o resultado da operação de consulta.

Para resolver esse problema, é essencial adotar uma solução que não trave a thread, permitindo que ela seja liberada para atender outras requisições ou executar outras tarefas importantes enquanto aguarda a conclusão da pesquisa.É neste momento que entramos na programação reativa, uma abordagem mais eficiente e que ajuda a eliminar esse gargalo.

A **programação reativa** propõe uma abordagem diferente, focada na forma como as aplicações lidam com dados e eventos em tempo real, especialmente em sistemas que exigem alta escalabilidade e responsividade.

Nesse paradigma, o foco está em **responder a eventos** de forma assíncrona e não bloqueante, permitindo que as threads sejam liberadas para realizar outras operações enquanto uma tarefa está em execução.

Esse modelo pode ser comparado a um sistema de notificação: ao invés de constantemente verificar se uma operação foi concluída (como na abordagem imperativa), a programação reativa permite que o sistema continue executando outras tarefas e receba uma notificação quando o resultado estiver disponível. Isso elimina ociosidade e melhora o desempenho, especialmente em sistemas com alta concorrência.

Para entender melhor como funciona a programação reativa, vejamos algumas de suas **principais características**:

- **Orientação a eventos:** as operações são disparadas por eventos, como recebimento de mensagens, alterações de estado ou respostas de sistemas externos. Isso torna a execução dinâmica e eficiente.
- **Execução assíncrona e não bloqueante:** ao contrário da programação imperativa, tarefas não bloqueiam threads enquanto esperam pela

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

conclusão de operações longas, como chamadas de APIs ou consultas a bancos de dados.

- **Fluxos de dados contínuos:** dados podem ser processados como fluxos contínuos, onde cada evento ou dado recebido é tratado imediatamente.
- **Desacoplamento:** componentes reativos interagempor meio de mensagens ou fluxos, sem depender diretamente uns dos outros, o que facilita a manutenção e a escalabilidade do sistema.

BENEFÍCIOS DA PROGRAMAÇÃO REATIVA

1. **Escalabilidade**: com threads liberadas para atender múltiplas requisições, sistemas reativos são capazes de lidar com maior número de conexões simultâneas.
2. **Responsividade**: as aplicações reativas podem responder rapidamente a eventos e oferecer uma experiência de usuário melhorada, mesmo sob alta carga.
3. **Melhor utilização de recursos**: reduz o desperdício de recursos, já que threads não ficam ociosas enquanto esperam por operações longas.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, aprendemos sobre programação imperativa e reativa, explorando seus conceitos, diferenças e casos de uso. Vimos como a programação imperativa se baseia em um fluxo sequencial de comandos, enquanto a programação reativa é orientada a eventos, focando em trabalhar com fluxos de dados de forma assíncrona e responsiva.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

BONER, J.; KUHN, J. **Reactive Microsystems: The Evolution ofMicroservicesatScale.**Sebastopol: O'Reilly Media, 2018.

NURKLEWICZ, T. **ReactiveProgrammingwithRxJava**. Beijing: O'Reilly Media, 2016.

SHARMA, R. **ReactiveProgrammingwithReactor 3**. Birmingham: PacktPublishing, 2018.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Programação Reativa, Programação Imperativa, Quarkus.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com